ServoControlWpf 소스 코드 설명

클래스별 역할 · 서로 어떻게 협력하는가 · 어디를 고치면 되는가 — 소스를 처음 여는 사람을 위한 안내

작성 정재현 (Claude Code 와 함께 작성) · 2026. 10. 26. · Rev. 1.0 · 함께 보는 문서: *ServoControlWpf 설계 과정과 운용 원리*(왜 이렇게 만들었나), 이 문서(코드가 어떻게 생겼나)

> **이 문서를 읽는 법**
>
> 소스 파일 하나하나를 "**무슨 일을 맡은 부품인가**" 관점으로 설명합니다. 각 클래스마다 ① 한 줄 역할 ② 쉽게 말하면(비유) ③ 주요 멤버 ④ 누구와 협력하나 ⑤ 주의점 순서입니다. 바쁘면 **2장 클래스 지도**와 **9장 시나리오**만 봐도 전체가 잡힙니다. 코드를 고치고 싶으면 **10장 "어디를 고치면 되나"** 표를 보세요.

차례

1. 읽기 전에 — 네 개의 층
2. 한눈에 보는 클래스 지도
3. Model 층 — 서보와 통신을 아는 부품
4. Services 층 — 바깥 프로그램을 부르는 부품
5. ViewModel 층 — 화면과 모델 사이의 통역
6. View 층 — 눈에 보이는 것
7. 에뮬레이터와 도구 — Pi 없이 돌리기
8. 테스트 프로젝트 — 무엇을 지키는가
9. 시나리오로 따라가기 — 버튼 하나가 지나가는 길
10. 어디를 고치면 되나
11. 용어 사전

1. 읽기 전에 — 네 개의 층

이 프로그램은 **MVVM**(Model · View · ViewModel) 구조에 **Services** 를 더한 네 개의 층으로 나뉩니다. 층을 나눈 이유는 간단합니다. **"서보를 아는 코드"** 와 **"화면을 아는 코드"** 를 섞지 않으면, 화면 없이도 서보 코드를 테스트할 수 있고, 통신 방식이 바뀌어도 화면을 고칠 필요가 없기 때문입니다.

층	폴더	맡은 일	비유
Model	`ServoControlWpf\Model\`	서보 사양·상태, pigpiod 와의 TCP 통신, WSL 터널, 설정 파일. **화면을 전혀 모른다.**	엔진과 배선
Services	`ServoControlWpf\Services\`	`wsl.exe` 같은 바깥 프로그램을 실행하고 출력을 받아 오는 도우미	외부 업체에 전화 거는 비서
ViewModel	`ServoControlWpf\ViewModel\`	화면의 요청을 모델에 전달하고, 모델의 변화를 화면용 속성으로 바꿔 준다. **Qt 판의 Controller 에 해당.**	운전석 계기판과 페달 사이의 통역
View	`ServoControlWpf\View\`, `App.xaml`	창·다이얼·카메라 그림. 로직은 거의 없고 ViewModel 의 속성에 **바인딩**만 한다	계기판과 핸들

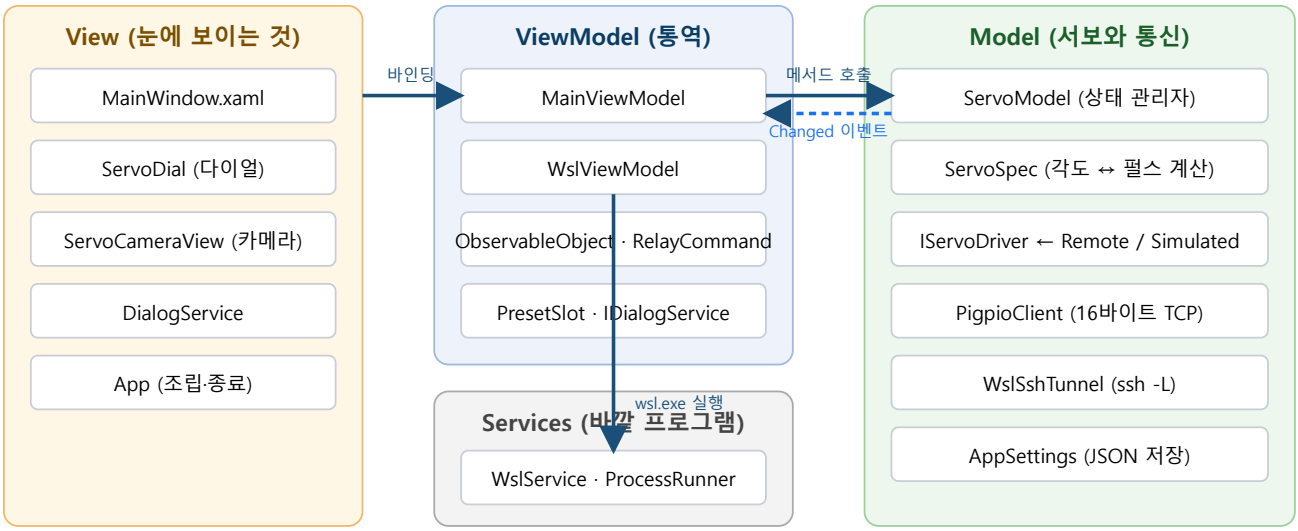


그림 1. 네 개의 층과 화살표. View 는 ViewModel 만, ViewModel 은 Model 과 Services 만 안다. Model 은 아무도 모른다(이벤트로만 알림).

왜 "아래 층은 위 층을 모르게" 하나

`ServoModel` 은 화면이 WPF 인지, 콘솔인지, 테스트인지 전혀 모릅니다. 그래서 테스트 프로젝트는 화면 없이 `ServoModel` 에 가짜 드라이버를 꽂아 "실패하면 이전 값을 유지하는가" 같은 규칙을 검사할 수 있습니다(8장). 반대로 `MainWindow.xaml` 은 TCP 가 뭔지 모릅니다. 연결 방식이 세 가지(직접·터널·에뮬레이터)여도 화면 코드는 한 줄도 다르지 않습니다.

2. 한눈에 보는 클래스 지도

프로젝트는 4개(앱 · 테스트 · 에뮬레이터 라이브러리 · 에뮬레이터 콘솔)이고, 앱 소스는 약 2,500줄입니다. 각 파일이 맡은 일을 한 줄로 적었습니다.

파일	줄	한 줄 역할	Qt 판 대응
Model — 서보와 통신			
`Model\ServoSpec.cs`	66	서보 사양(펄스 3개·각도 2개·GPIO)과 **각도 ↔ 펄스폭 변환**	`servospec.h`
`Model\IServoDriver.cs`	17	"펄스를 내보내는 장치" 의 공통 규격(인터페이스)	`servodriver.h`
`Model\ServoException.cs`	15	드라이버 실패를 알리는 예외. `LinkLost` 로 "통신이 끊긴 것인지" 표시	`bool + QString*`
`Model\SimulatedDriver.cs`	14	아무것도 안 보내는 가짜 드라이버 (연결 전·테스트용)	`simulateddriver.h`
`Model\PigpioClient.cs`	118	pigpiod 의 **16바이트 TCP 규약**을 말하는 가장 낮은 층	`RemotePigpioDriver::command()`
`Model\RemotePigpioDriver.cs`	64	PigpioClient 를 써서 실제 서보를 움직이는 드라이버. 직접·터널·에뮬레이터 모두 이 클래스 하나	`remotepigpiodriver.*`
`Model\WslSshTunnel.cs`	158	`wsl.exe -e ssh -L` 를 띄워 Pi 의 8888 포트를 127.0.0.1:18888 로 끌어오는 터널	(신규)
`Model\ServoModel.cs`	99	**현재 각도·출력 여부·사양을 기억하는 상태 관리자.** 바뀌면 `Changed` 이벤트	`servomodel.*`
`Model\AppSettings.cs`	98	설정(주소·모드·보정값·위치 메모리)을 JSON 파일로 저장·복원	`QSettings`
Services — 바깥 프로그램			
`Services\ProcessRunner.cs`	67	프로그램을 창 없이 실행하고 출력을 한 줄씩 받아 오는 도우미	`QProcess`
`Services\WslService.cs`	126	`wsl.exe` 전용 창구: 배포판 확인, 스크립트 실행, ssh 명령, 터미널 창 열기	(신규)
ViewModel — 통역			
`ViewModel\ObservableObject.cs`	19	"값이 바뀌었다" 고 화면에 알리는 기능 (`INotifyPropertyChanged`) 의 밑바탕	Qt signal
`ViewModel\RelayCommand.cs`	21	버튼·단축키를 ViewModel 메서드에 묶는 `ICommand`	`connect(signal, slot)`
`ViewModel\IDialogService.cs`	8	ViewModel 이 MessageBox 를 직접 부르지 않게 하는 작은 창구	(신규)

`ViewModel\PresetSlot.cs`	27	위치 메모리 한 칸(P1~P4)	`QSettings` 일부
`ViewModel\MainViewModel.cs`	490	**프로그램의 두뇌.** 연결·각도·펄스·스윕·메모리·보정·설정 저장을 모두 조율	`servocontroller.*`
`ViewModel\WslViewModel.cs`	236	"WSL / Pi 관리" 탭: 스크립트·ssh 실행과 로그	(신규)
View — 눈에 보이는 것			
`App.xaml(.cs)`	60	시작할 때 부품을 조립하고, 끝날 때 서보 출력을 끈다	`main.cpp`
`View\MainWindow.xaml(.cs)`	309+68	창 배치와 바인딩. 코드 비하인드는 전체화면·Enter·휠 같은 순수 화면 동작만	`mainwindow.*`
`View\ServoDial.cs`	236	직접 그린 원형 다이얼(노브). 드래그·휠·키보드·UI 자동화 지원	`QDial`
`View\ServoCameraView.cs`	271	서보에 올린 카메라를 위에서 본 그림. 연결됐을 때만 서보 속도로 회전	`servogauge.*` (역할 변경)
`View\DialogService.cs`	21	MessageBox · 폴더 선택 창 (IDialogService 구현)	`QMessageBox`
에뮬레이터 · 도구 · 테스트			
`PigpioEmulator\PigpioEmulatorServer.cs`	182	가짜 pigpiod 라이브러리. 같은 16바이트 규약 + 서보 움직임 흉내	(신규)
`PigpioEmulatorCli\Program.cs`	62	가짜 pigpiod 콘솔 프로그램 `pigpio-emu`	(신규)
`tools\fake_pigpiod.py`	94	리눅스(WSL)에서 돌리는 가짜 pigpiod (파이썬 표준 라이브러리만)	(신규)
`run-from-wsl.sh` · `Directory.Build.targets`	45+12	WSL 셸에서 Windows 앱 띄우기 · 리눅스 dotnet 으로 WPF 빌드 시 한글 안내	(신규)
`ServoControlWpf.Tests\*.cs` (8개)	~640	xUnit 테스트 53개	(신규)

3. Model 층 — 서보와 통신을 아는 부품

ServoSpec Model₩ServoSpec.cs · record struct

> **역할:** 서보 한 개의 "사양표" 와 **각도 ↔ 펄스폭 환산**.

쉽게 말하면 온도계의 눈금표입니다. "1500 µs 가 0°, 1900 µs 가 +40°" 라는 눈금을 들고 있다가, 각도를 주면 펄스폭을, 펄스폭을 주면 각도를 계산해 줍니다. 값이 바뀌지 않는 `record struct` 라서 복사해도 안전하고, 보정값을 바꿀 때는 새 `ServoSpec` 을 만들어 통째로 갈아 끼웁니다.

멤버	하는 일
`MinPulse / NeutralPulse / MaxPulse`	최소·중립·최대 펄스폭(µs). 기본 1100 / 1500 / 1900
`MinAngle / MaxAngle / Gpio`	허용 각도(기본 ±40°)와 BCM 핀 번호(기본 18)
`Hs311Official()` · `Hs311Extended()`	제조사 공식값(±40°) · 비공식 확장값(600~2400 µs, ±90°)
`AngleToPulse(angle)`	중립을 기준으로 **+쪽과 −쪽을 따로** 비례 계산. 양쪽 기울기가 달라도(예 1100/1500/2100) 보정이 됨. 범위 밖 각도는 먼저 잘라 냄
`PulseToAngle(pulse)`	반대 방향 변환. 펄스 슬라이더·입력칸이 각도를 바꿀 때 사용
`IsValid()`	pigpio 허용 범위(500~2500), 최소<중립<최대, ±90° 이내, GPIO 2~27 검사. 보정값 적용 전에 반드시 통과해야 함

협력자: `ServoModel`(현재 사양 보관), `MainViewModel`(펄스 ↔ 각도 변환), `AppSettings`(저장·복원). **주의:** 10 µs/° 라는 숫자가 코드에 직접 박혀 있지 않습니다. 모두 사양표로부터 계산되므로 다른 서보를 쓰려면 사양값만 바꾸면 됩니다.

IServoDriver Model₩IServoDriver.cs · interface · ServoException Model₩ServoException.cs

> **역할:** "펄스를 내보내는 장치" 라면 누구나 지켜야 하는 규격 + 실패를 알리는 예외.

쉽게 말하면 콘센트 규격입니다. `ServoModel` 은 콘센트(`IServoDriver`)에만 꽂고, 뒤에 실제 Pi 가 있든(`RemotePigpioDriver`) 아무것도 없든(`SimulatedDriver`) 신경 쓰지 않습니다. 이것이 **Strategy 패턴**이고, [연결]/[연결 해제] 는 사실 "드라이버 갈아 끼우기" 입니다.

멤버	하는 일
`Name` · `Info`	상태 표시용 이름("에뮬레이터 127.0.0.1:8888 · GPIO18")과 장치 정보("Pi 리비전 c03111 · pigpio v79")
`IsRemote` · `IsLinkAlive`	원격 장치인지 · 통신 경로가 살아 있는지. ViewModel 의 "연결됨" 표시와 자동 해제 판단에 쓰임
`Open(gpio)`	핀 준비. 원격이면 이때 TCP 접속
`WritePulse(µs)` · `Release()`	펄스 출력 · 펄스 0(힘 빼기)
`ServoException.LinkLost`	**true** = 통신이 끊긴 오류(연결 정리 필요) / **false** = pigpio 가 명령만 거부(연결 유지). ViewModel 의 `HandleFailure` 가 이 값으로 갈림

주의: Qt 판은 `bool` 반환 + 오류 문자열 포인터였지만, C# 판은 **예외**로 바꿨습니다. 덕분에 "실패하면 상태를 바꾸지 않는다" 는 규칙을 `try/catch` 한 줄로 지킬 수 있습니다.

SimulatedDriver Model₩SimulatedDriver.cs

> **역할:** 아무 데도 보내지 않는 빈 드라이버. 프로그램이 처음 켜졌을 때와 [연결 해제] 뒤에 꽂혀 있다.

쉽게 말하면 플러그를 뽑아 둔 상태를 나타내는 "빈 콘센트 마개" 입니다. 14줄뿐이지만 덕분에 `ServoModel` 은 "드라이버가 null 인가" 를 어디서도 검사하지 않아도 됩니다(Null Object 패턴). 화면은 그대로 동작하므로 Pi 없이도 다이얼·슬라이더를 연습할 수 있습니다.

PigpioClient Model₩PigpioClient.cs

> **역할:** pigpiod 와 주고받는 **16바이트 명령**을 만들고 읽는, 가장 낮은 통신 층.

쉽게 말하면 외국어 통역사입니다. "GPIO18 을 1700 µs 로" 라는 요청을 pigpiod 가 알아듣는 16바이트(`cmd=8, p1=18, p2=1700, p3=0`, 각각 4바이트 little-endian)로 바꿔 보내고, 돌아온 16바이트의 마지막 4바이트(`res`)를 읽어 성공(0 이상)인지 오류(음수)인지 알려 줍니다.

멤버	하는 일
`CmdServo=8 · CmdHwVer=17 · CmdPigpv=26`	쓰는 명령 번호 세 개 (서보 펄스 · 하드웨어 리비전 · pigpio 버전)
`Connect(timeoutMs)`	TCP 접속. 1.5초 안에 안 되면 `ServoException(LinkLost)`. `NoDelay=true` 로 명령마다 바로 전송
`Command(cmd, p1, p2)`	요청 16바이트 쓰기 → 응답 16바이트가 다 올 때까지 읽기 → 응답의 cmd 가 요청과 같은지 확인 → `res` 반환. 시간 초과·끊김은 `LinkLost=true`, 음수 코드는 `LinkLost=false`
`ErrorText(code)`	−2 → "사용할 수 없는 GPIO", −7 → "펄스폭은 0 또는 500~2500", −41 → "권한 없음" 처럼 한글로
`Abort()` · `IsConnected`	오류 뒤 소켓 정리 · 접속 여부

협력자: `RemotePigpioDriver` 만 이 클래스를 씁니다. 따로 떼어 놓은 이유는 **테스트** 때문입니다. `PigpioClientTests` 가 가짜 pigpiod(`TcpListener`)를 띄워 "정확히 이 16바이트가 나가는가" 를 검사합니다. **주의:** PC 에 pigpio 라이브러리를 설치할 필요가 없습니다. `TcpClient` 와 `BinaryPrimitives` 만 씁니다.

RemotePigpioDriver Model₩RemotePigpioDriver.cs · IServoDriver 구현

> **역할:** 실제(또는 가짜) Pi 의 서보를 움직이는 드라이버. **직접 연결·WSL 터널·에뮬레이터 세 모드가 모두 이 클래스 하나**로 처리된다.

쉽게 말하면 리모컨입니다. 어느 주소(`host:port`)로 보낼지만 다를 뿐 누르는 버튼은 같습니다. 직접 연결은 `192.168.10.2:8888`, 터널은 `127.0.0.1:18888`, 에뮬레이터는 `127.0.0.1:8888`.

멤버	하는 일
`Open(gpio)`	접속 후 HWVER·PIGPV 두 명령으로 상대가 진짜 pigpiod 인지 확인하고 `Info`("Pi 리비전 … · pigpio v…")를 채움. 핀이 바뀌면 이전 핀을 먼저 끔
`WritePulse(µs)`	`SERVO(gpio, µs)` 한 줄
`Release()`	펄스 0. **통신이 끊긴 뒤라도 0.5초 재접속해서 끔** — pigpiod 는 접속이 끊겨도 마지막 펄스를 계속 내보내기 때문
`Dispose()`	Release → 소켓 닫기 → 함께 받은 `_tunnel`(터널 또는 에뮬레이터 서버) 닫기

주의: 생성자의 `IDisposable? tunnel` 자리에 터널 모드는 `WslSshTunnel`, 에뮬레이터 모드는 `PigpioEmulatorServer` 가 들어옵니다. 드라이버가 사라질 때 그것도 같이 정리되므로 "연결 해제 = 터널·가짜 서버도 종료" 가 자동으로 지켜집니다.

WslSshTunnel Model₩WslSshTunnel.cs

> **역할:** "WSL 터널" 모드의 핵심. `wsl.exe -e ssh -N -L 127.0.0.1:18888:127.0.0.1:8888 rpi` 를 띄우고 포트가 열릴 때까지 기다린다.

쉽게 말하면 지하 통로를 파는 공사반입니다. 통로가 뚫리면(127.0.0.1:18888 이 열리면) 그 뒤로는 `RemotePigpioDriver` 가 그 통로로 평소처럼 명령을 보냅니다. WSL2 가 WSL 안의 127.0.0.1 포트를 Windows 쪽 127.0.0.1 로도 열어 주기 때문에(localhost 포워딩) 이 방법이 됩니다.

멤버	하는 일
`OpenAsync(distro, alias, localPort, …)`	① 포트가 이미 열려 있으면 기존 터널 재사용 → ② `WslService.ProbeAsync` 로 WSL·배포판 확인 → ③ ssh 프로세스 시작(BatchMode, ExitOnForwardFailure, ConnectTimeout 5) → ④ 최대 10초 동안 200 ms 마다 포트 확인. ssh 가 먼저 죽으면 stderr 를 붙여 `ServoException`
`Dispose()`	우리가 띄운 wsl.exe 를 프로세스 트리째 종료(`Kill(entireProcessTree)`). 재사용한 터널이면 건드리지 않음
`Describe()`	"WSL 터널 → rpi (Ubuntu)" 같은 표시 문자열

주의: 종료 코드는 `Dispose()` **전에** 읽어야 합니다(뒤에 읽으면 예외). 이 순서 때문에 한 번 "연결 중…" 에서 멈추는 버그가 있었고, `WslSshTunnelTests` 가 재발을 막습니다. ssh 는 비밀번호를 물을 수 없으므로(BatchMode) **키 인증**이 전제입니다.

ServoModel Model₩ServoModel.cs

> **역할:** **"지금 서보가 어떤 상태인가"** 를 기억하는 단 하나의 장소. 각도·출력 여부·사양·드라이버를 들고 있고, 바뀔 때마다 `Changed` 이벤트를 낸다.

쉽게 말하면 관제탑의 현황판입니다. 모든 요청(각도 바꾸기, 출력 끄기, 보정값 바꾸기, 드라이버 교체)은 여기를 거치고, 현황판이 바뀌면 구독자(`MainViewModel`)에게 알립니다. **기억하는 것은 "각도" 하나**이고 펄스폭은 `Spec.AngleToPulse(Angle)` 로 그때그때 계산합니다. 그래서 각도 줄·다이얼·펄스 줄·카메라가 어긋날 수 없습니다.

멤버	하는 일
`Angle` · `Pulse` · `IsOutputOn` · `Spec`	현재 상태. `Pulse` 는 계산 속성
`IsConnected` · `IsLinkAlive` · `DriverName` · `DriverInfo`	드라이버에게 물어서 그대로 전달
`SetAngle(deg)`	범위로 자른 뒤 드라이버에 펄스 출력 → **성공했을 때만** Angle 저장, 출력 켜짐 표시. 실패하면 예외만 던지고 상태는 그대로
`Release()`	드라이버 Release → 출력 꺼짐 표시
`SetSpec(spec)`	유효성 검사 → GPIO 가 바뀌면 이전 핀 끄고 새 핀 열기(실패 시 이전 핀 복구) → 출력 중이면 새 사양으로 먼저 써 보고 성공해야 반영. **실패 시 이전 사양 유지**
`SetDriver(driver)`	이전 드라이버 Release·Dispose(접속·터널 종료) 후 교체. [연결]/[연결 해제] 가 이것
`Dispose()`	드라이버 Dispose(펄스 0 전송 포함). 창을 닫을 때 호출

주의: 이 클래스는 WPF 를 전혀 참조하지 않습니다(`using System.Windows` 없음). 그래서 `ServoModelTests` 가 가짜 드라이버만으로 "실패하면 되돌리는가" 를 검사할 수 있습니다.

AppSettings Model₩AppSettings.cs · ConnectionMode enum

> **역할:** 설정을 `%APPDATA%\VI\ServoControl.json` 에 저장·복원. `ConnectionMode` 는 직접(Direct)·터널(WslTunnel)·에뮬레이터(Emulator) 세 값.

쉽게 말하면 프로그램의 수첩입니다. 주소·포트·모드·ssh 별명·터널 포트·스크립트 폴더·보정값 6개·위치 메모리 4칸·스윕 속도를 적어 두고, 다음에 켤 때 그대로 꺼냅니다.

멤버	하는 일
`Load()` · `Save()`	JSON 읽기·쓰기(`System.Text.Json`, 들여쓰기, enum 은 문자열). 파일이 손상되면 기본값으로 시작하고, 저장 실패는 무시(치명적이지 않음)
`LoadSpec()` · `StoreSpec(spec)`	저장된 보정값 → `ServoSpec`. 값이 이상하면 HS-311 공식값으로(GPIO 번호는 유지)
`FilePath`	저장 위치. Qt 판의 `~/.config/VI/ServoControl.conf` 에 해당

주의: 이 PC 에 예전 보정값(800/1500/2100 µs)이 저장돼 있으면 1234 µs 가 −15.2° 로 보이는 식으로 계산이 달라집니다. 이상하면 이 파일을 지우고 다시 켜면 공식값으로 돌아옵니다.

4. Services 층 — 바깥 프로그램을 부르는 부품

ProcessRunner Services₩ProcessRunner.cs · static

> **역할:** 아무 프로그램이나 **창 없이** 실행하고, 출력을 **한 줄씩 실시간으로** 받아 오며, 취소하면 프로세스 트리째 죽이는 도우미.

쉽게 말하면 심부름꾼입니다. "이 프로그램 돌리고 뭐라고 하는지 한 줄씩 알려 줘" 라고 시키면 `onLine(줄, 오류 출력인지)` 로 보고합니다.

멤버	하는 일
`RunAsync(file, args, encoding, onLine, ct)`	`ProcessStartInfo` 로 실행(표준 입력은 바로 닫아 입력 대기로 멈추지 않게) → stdout/stderr 를 줄 단위로 이벤트 수신 → 끝나면 `Result(ExitCode, StdOut, StdErr)`

주의: 인코딩을 매개변수로 받는 이유는 `wsl -l` 같은 관리 명령은 **UTF-16LE**, 리눅스 명령 출력은 **UTF-8** 이기 때문입니다. 틀리면 글자가 깨집니다.

WslService Services₩WslService.cs · static

> **역할:** WPF 가 WSL(리눅스)에 접근하는 **유일한 창구**. 모든 `wsl.exe` 호출은 여기서 만들어진다.

멤버	하는 일
`ProbeAsync()`	`wsl -l -q` 로 WSL 설치 여부와 배포판 목록 확인(UTF-16). 없으면 "wsl --install 로 설치하세요" 안내
`RunScriptAsync(distro, dir, script, args, …)`	`wsl [-d 배포판] --cd 폴더 -e bash ./스크립트`. `--cd` 가 Windows 경로(D:₩…)를 /mnt/d/… 로 자동 변환하고 UNC(₩₩wsl.localhost₩…)도 받아들임
`RunSshAsync(distro, alias, cmd, …)`	`wsl -e ssh -o BatchMode=yes rpi "명령"`. Pi 상태 확인·자유 명령에 사용
`RunShellAsync(distro, dir, cmd, …)`	임의 리눅스 명령(예 `cat pi_info.txt`)
`OpenScriptInTerminal(…)`	`pi_setup.sh` 처럼 **sudo 비밀번호를 물을 수 있는** 스크립트를 **보이는 콘솔 창**에서 실행. GUI 가 콘솔 프로그램을 창 숨김 없이 띄우면 Windows 가 새 콘솔을 만들어 줌. 끝나면 "아무 키나 누르면 닫힘"

협력자: `WslViewModel`(탭의 버튼들), `WslSshTunnel`(Probe 로 사전 확인). **주의:** 모든 메서드가 `static` 입니다. 상태가 없어서 어디서든 바로 부를 수 있지만, 테스트에서 가짜로 바꿔 끼우기는 어렵습니다(그래서 WSL 관련 테스트는 실제 wsl.exe 를 호출합니다).

5. ViewModel 층 — 화면과 모델 사이의 통역

ObservableObject ViewModel₩ObservableObject.cs · abstract · **RelayCommand** ViewModel₩RelayCommand.cs

> **역할:** MVVM 의 두 가지 기본 도구. 외부 패키지(MVVM Toolkit 등) 없이 직접 구현했다.

- **ObservableObject** — 속성이 바뀌면 `PropertyChanged` 이벤트로 화면에 알립니다. `Set(ref field, value)` 는 "값이 실제로 바뀌었을 때만 저장하고 알림" 을 한 줄로 해 줍니다. Qt 의 signal 과 같은 역할.
- **RelayCommand** — 버튼의 `Command="{Binding XxxCommand}"` 가 ViewModel 의 메서드를 부르게 하는 `ICommand`. `canExecute` 로 버튼 활성/비활성을 결정하고, `RaiseCanExecuteChanged()` 로 다시 계산시킵니다. Qt 의 `connect(signal, slot)` 에 해당.

IDialogService ViewModel₩IDialogService.cs · **PresetSlot** ViewModel₩PresetSlot.cs

- **IDialogService** — `ShowError`, `Confirm`, `PickFolder` 세 가지. ViewModel 이 `MessageBox` 를 직접 부르면 테스트에서 창이 떠 버리므로, 인터페이스로 빼고 테스트에서는 가짜(`FakeDialogs`)를 꽂습니다.
- **PresetSlot** — 위치 메모리 한 칸. `Angle`(null 이면 빈 칸)과 버튼에 보일 `Label`("P1₩n+11.0°" 또는 "P1₩n(비어 있음)").

MainViewModel ViewModel₩MainViewModel.cs · 490줄

> **역할: 프로그램의 두뇌.** 화면의 모든 요청을 받아 모델을 조작하고, 모델의 변화를 화면용 속성으로 바꿔 준다. Qt 판의 `ServoController` 와 View 의 상태를 합친 것.

쉽게 말하면 운전석의 통역사입니다. 핸들(다이얼)을 돌리면 엔진(ServoModel)에 "이 각도로" 라고 전하고, 엔진이 "지금 20° 야" 라고 하면 계기판(바인딩된 속성들)을 고쳐 씁니다.

속성 묶음

속성	뜻
`Host · Port · ModeIndex · TunnelLocalPort`	연결 설정. `ModeIndex` 0/1/2 = 직접/터널/에뮬레이터(ComboBox 와 바인딩), `IsDirectMode/IsTunnelMode/IsEmulatorMode` 로 입력칸을 보였다 숨김
`IsConnected · IsBusy · CanEditConnection · ConnectButtonText`	연결 상태. 연결 중에는 버튼이 "연결 중…" 이 되고 설정칸이 잠김
`Angle`	다이얼·각도 슬라이더·각도 입력칸과 **양방향** 바인딩. 화면에서 set 하면 `RequestAngle` → 모델; 모델에서 온 갱신(`_syncing=true`)이면 저장만
`PulseInput · MinPulse · MaxPulse`	펄스 슬라이더·입력칸과 양방향. set 하면 범위로 자른 뒤 `Spec.PulseToAngle` 로 각도를 구해 이동 → 각도 줄도 따라 바뀜
`Pulse · IsOutputOn · OutputText · AngleBigText`	모델에서 읽기만 하는 표시용
`IsSweeping · SweepSpeed · Presets · Cal*`	스윕, 위치 메모리 4칸, 보정 입력칸 6개
`EmulatorText · StatusText`	가짜 Pi 가 받은 마지막 명령("가짜 Pi 수신 #3: SERVO gpio=18 1700 µs") · 4초 뒤 사라지는 상태 줄
`Wsl`	WSL 탭용 `WslViewModel`

커맨드(버튼·단축키)

커맨드	하는 일
`ConnectToggleCommand`	연결/해제 토글. 모드에 따라 ① 터널: `WslSshTunnel.OpenAsync` 후 `RemotePigpioDriver(127.0.0.1, 18888, tunnel)` ② 에뮬레이터: `StartEmulatorDriver()` ③ 직접: `RemotePigpioDriver(Host, Port)`. `driver.Open` 은 `Task.Run` 으로 돌려 1.5초 타임아웃 동안 화면이 멈추지 않게. 실패하면 드라이버를 버리고 오류 창
`SetAngle · Nudge(±1/±10) · Center · GoMin · GoMax`	모두 `RequestAngle(각도)` 로 모임
`ReleaseCommand` (Space)	스윕 정지 → `model.Release()`
`ToggleSweepCommand`	20 ms 타이머로 `angle += 방향 × 속도 × 0.02`, 끝에서 방향 반전. 손으로 조작하면 자동 정지
`PresetRecall/Store(i)`	메모리 이동/저장(저장 즉시 `AppSettings.Save`)
`ApplySpec · ApplyOfficial · ApplyExtended`	보정값 적용. 확장값은 먼저 `Confirm` 경고. 실패하면 입력칸을 실제 적용값으로 되돌림

핵심 내부 메서드

메서드	하는 일
`RequestAngle(deg)`	스윕 정지 → `model.SetAngle` → 실패면 `HandleFailure`
`HandleFailure(ex)`	연결돼 있고 `LinkLost`(또는 링크 죽음)면 **먼저 연결 해제**(SimulatedDriver 로 교체) 후 "연결을 해제했습니다. 다시 [연결]을..." 안내. pigpio 가 거부만 한 오류면 연결 유지하고 메시지만
`SyncFromModel()`	모델 → 화면. `_syncing=true` 로 감싸서 `Angle` setter 가 다시 명령을 보내는 **무한 루프를 차단**(Qt 의 `QSignalBlocker` 역할)
`StartEmulatorDriver()`	앱 안에 `PigpioEmulatorServer("127.0.0.1", Port)` 를 띄우고 실제와 똑같은 `RemotePigpioDriver` 로 접속. 서버의 `CommandReceived` 를 UI 스레드로 넘겨 `EmulatorText` 갱신
`Shutdown()`	창 닫을 때: 스윕 정지 → 설정 저장 → WSL 작업 취소 → `model.Dispose()`(펄스 0, 터널 종료)

주의: 서보 명령은 Qt 판처럼 **동기**(1.5초 타임아웃)로 단순하게 두었습니다. 비동기는 연결 과정(터널 대기)에만 씁니다. `catch (Exception)` 으로 예상 못 한 오류까지 잡아 "연결 중 예상하지 못한 오류" 창으로 보여 주므로 앱이 조용히 멈추지 않습니다.

WslViewModel ViewModel₩WslViewModel.cs

> **역할:** "WSL / Pi 관리" 탭. 배포판·ssh 별명·스크립트 폴더 설정, 버튼 7개, 로그 창.

커맨드	하는 일
`ProbeCommand` [WSL 확인]	`WslService.ProbeAsync` → "WSL 사용 가능 · 배포판 1개", 목록을 콤보박스에
`CheckPiCommand` [Pi 상태 확인]	ssh 로 hostname·IP·pigpiod 상태·온도·가동 시간·8888 접속 수를 한 번에
`RunPiInfoCommand` · `RunDeployCommand`	기존 Qt 프로젝트의 `pi_info.sh`(끝나면 `pi_info.txt` 내용 표시) · `deploy.sh` 를 인라인 실행
`OpenSetupTerminalCommand` [pi_setup.sh ↗]	sudo 가 필요하므로 **새 터미널 창**에서
`RunSshCommand` · `CancelCommand` · `ClearLogCommand` · `BrowseScriptDirCommand`	자유 ssh 명령 · 실행 취소 · 로그 지우기 · 폴더 선택

핵심 메서드 `RunGuardedAsync`: 모든 실행을 감싸서 "실행 중" 표시(버튼 잠금), 취소 토큰, 종료 코드·실패·취소 로그를 한곳에서 처리합니다. 출력 줄은 백그라운드 스레드에서 오므로 `SynchronizationContext.Post` 로 UI 스레드에 넘겨 `Log` 에 붙입니다(2000줄 넘으면 앞을 버림). **주의:** 스크립트 폴더가 `/` 나 `~` 로 시작하면 WSL 내부 경로로 보고 존재 여부를 Windows 에서 확인하지 않습니다.

6. View 층 — 눈에 보이는 것

App App.xaml / App.xaml.cs

> **역할:** 시작할 때 부품을 **조립**하고(Composition Root), 끝날 때 **서보 출력을 끈다**. Qt 판의 `main.cpp`.

```
var settings = AppSettings.Load();              // 수첩 꺼내기
var driver = new SimulatedDriver();             // 처음엔 빈 드라이버
var model = new ServoModel(driver);  model.Start(settings.LoadSpec());
var window = new MainWindow();
_vm = new MainViewModel(model, settings, new DialogService(window));
window.DataContext = _vm;                       // 바인딩의 출발점
window.Closed += (_, _) => ShutdownOnce();      // 창 닫힘 → 펄스 0, 터널 종료, 설정 저장
```

`App.xaml` 에는 공통 스타일(제목 글꼴, 빨간 "출력 끄기" 버튼, 초록 "연결됨" 버튼 등)이 있습니다. `DispatcherUnhandledException` 을 잡아 예상 못 한 오류가 나도 메시지를 보여 준 뒤 출력을 끄고 종료합니다. **주의:** `ShutdownOnce` 는 `Closed` 와 `OnExit` 두 곳에서 불릴 수 있어 `_vm = null` 로 한 번만 실행되게 막습니다.

MainWindow View₩MainWindow.xaml (309줄) + MainWindow.xaml.cs (68줄)

> **역할:** 창의 배치와 바인딩. 코드 비하인드에는 "순수 화면 동작" 만 둔다.

XAML 배치: 상단 바(제목 · 연결 방식 콤보 · 모드별 입력칸 · [연결] · 상태 점 · 장치 정보 · [전체화면] [종료]) / 왼쪽 `ServoCameraView` / 가운데 "다이얼 조종"(`ServoDial`, 각도 줄, 펄스 줄, ±버튼, 최소/최대, 출력 끄기) / 오른쪽 탭(서보: 메모리·스윕·보정, WSL / Pi 관리) / 하단 상태 줄. 모든 값은 `{Binding …}` 으로 `MainViewModel` 에 묶여 있고, 버튼은 `Command`, 키보드는 `InputBindings` (←/→ 1°, Space 출력 끄기, F11 전체화면)입니다.

코드 비하인드	하는 일
`ToggleFullScreen()`	F11. 창 테두리 없는 최대화 ↔ 복귀(Maximized 상태에서 바로 None 으로 바꾸면 작업 표시줄이 남는 문제를 Normal 을 거쳐 회피)
`OnInputBoxKeyDown`	각도·펄스 입력칸에서 Enter → 바인딩 소스 갱신(= 이동) → 포커스를 다이얼로
`OnSliderWheel` · `OnPulseSliderWheel`	슬라이더 위 휠 한 칸 = 1° / 10 µs
`OnSshBoxKeyDown` · `OnLogChanged`	ssh 입력칸 Enter 실행 · 로그 자동 스크롤

ServoDial View₩ServoDial.cs · FrameworkElement

> **역할:** WPF 에 없는 원형 다이얼을 **직접 그린** 컨트롤. Qt 의 `QDial` 에 해당.

쉽게 말하면 볼륨 노브입니다. `Minimum~Maximum` 값을 화면 각도 −135°~+135°(270° 스윕, 위가 0)에 펼쳐 놓고, 마우스 방향을 `atan2` 로 각도로 바꿔 값을 정합니다.

멤버	하는 일
`Value` (양방향 기본) · `Minimum` · `Maximum` · `Step` · `IsOutputOn` · `Pulse`	의존 속성(DependencyProperty). `Value` 가 `MainViewModel.Angle` 과 묶임. `Pulse` 는 노브 안 힌트에 표시
`OnRender`	회색 궤적 → 0°부터 현재 값까지 파란 호 → 1°/10° 눈금과 숫자 → 그라데이션 노브 → 포인터 → 가운데 큰 값 + "1700 µs · 출력 켜짐"
`ValueFromPoint(pos)` · `Snap(v)`	마우스 위치 → 값(아래쪽 빈 구간은 가까운 끝으로) · `Step` (1°) 단위 맞춤과 범위 자름. `internal` 이라 테스트가 직접 호출
`OnMouse*` · `OnMouseWheel` · `OnKeyDown`	누르거나 끌면(`IsMouseCaptured` 동안) 값 변경, 휠 1칸 = 1°, ←/→ 1°, PgUp/PgDn 10°, Home/End 끝
`DialPeer` (AutomationPeer)	UI 자동화·접근성용. 이름 "각도 다이얼", `RangeValue` 패턴으로 값을 읽고 쓸 수 있어 자동 테스트와 화면 읽기 프로그램이 다룰 수 있음

ServoCameraView View₩ServoCameraView.cs · FrameworkElement

> **역할:** 서보에 올려진 카메라를 위에서 본 **표시 전용** 그림. 명령한 각도를 향해 HS-311 속도로 "실제로 돌아가는 모습" 을 보여 준다.

쉽게 말하면 CCTV 관제실의 미니어처입니다. 다이얼은 "가라고 한 곳", 이 그림은 "지금 가고 있는 곳" 입니다.

멤버	하는 일
`Angle` · `IsOutputOn` · `IsConnected` · `Pulse` · `MinAngle` · `MaxAngle` · `FieldOfView` · `SpeedDegPerSec`	의존 속성. `Angle` 은 목표(ViewModel 의 Angle, OneWay), 속도 기본 60°/0.19 s ≈ 316 °/s, 시야 62°
`_shown` · `ShownAngle` · `IsMoving` · `IsAnimating`	화면에 그려진 각도 · 목표와 다른지 · 타이머가 도는지
`IsDriven = IsConnected && IsOutputOn`	**구동 조건.** 거짓이면 어떤 값을 바꿔도 제자리. 연결이 없으면 서보로 명령이 가지 않으므로 화면도 움직이지 않는 것이 맞음
`UpdateAnimation()` · `OnTick()`	구동 조건을 만족하고 목표와 다르면 16 ms 타이머 시작 → 매 틱 `속도 × dt` 만큼 접근 → 도착하면 정지
`OnRender`	바닥 반원과 허용 범위 호 → 서보 본체(HS-311)와 케이블 → 시야 쐐기(구동 중일 때만) → 회전 변환 안에서 혼·카메라·렌즈 → 글자(현재 각도, "→ 목표 … 로 회전 중", 끝단 경고). **연결 해제면** 반투명 막을 덮고 "연결 안 됨 — [연결]하면 카메라 움직임이 표시됩니다" 만

주의: `IsHitTestVisible=false` 로 마우스를 전혀 받지 않습니다. 애니메이션은 화면 안에서만 일어나고 모델의 각도는 즉시 바뀝니다.

DialogService View₩DialogService.cs · IDialogService 구현

`MessageBox.Show`(경고 아이콘, 제목 "서보 제어") · 예/아니오 확인 · `OpenFolderDialog`. 창 소유자를 넘겨 주 창 위에 뜨게 합니다. 21줄짜리 클래스지만 이것 덕분에 ViewModel 이 WPF 대화상자를 모르고, 테스트는 가짜로 바꿔 끼웁니다.

7. 에뮬레이터와 도구 — Pi 없이 돌리기

PigpioEmulatorServer

PigpioEmulator₩PigpioEmulatorServer.cs · 별도 라이브러리 프로젝트

> **역할: 가짜 pigpiod.** 실제와 같은 16바이트 규약으로 응답하므로 WPF(와 Qt 판)는 진짜 Pi 인지 가짜인지 구별하지 못한다.

쉽게 말하면 운전 연습용 시뮬레이터입니다. TCP·바이트 열·오류 처리 경로를 실제와 똑같이 밟되, 끝에 서보 대신 숫자만 바뀝니다.

멤버	하는 일
`Start()` · `Dispose()`	`TcpListener` 로 듣기 시작(포트가 이미 쓰이면 `SocketException`) · 정지
`Handle(cmd, p1, p2)`	SERVO: gpio>31 → −2, 펄스가 0 도 500~2500 도 아니면 −7, 아니면 저장하고 0. HWVER → 0xc03111(Pi 4B), PIGPV → 79. 그 밖은 0
`GetSimulatedAngle(gpio)` · `AdvanceAngle`	목표 펄스를 향해 0.19 s/60° 속도로 움직이는 "현재 각도" 흉내(1500 µs = 0°, 10 µs/°). 출력 0 이면 그 자리에 멈춤
`CommandReceived` · `PulseChanged` · `ClientCountChanged`	이벤트(스레드 풀에서 발생). WPF 는 `CommandReceived` 로 "가짜 Pi 수신 #n: …" 을 표시

협력자: `MainViewModel.StartEmulatorDriver`(앱 안), `PigpioEmulatorCli`(콘솔), `EmulatorTests`. **주의:** WPF 를 참조하지 않는 순수 .NET 라이브러리라 **리눅스 dotnet 으로도 빌드·실행**됩니다.

PigpioEmulatorCli — Program.cs

콘솔 프로그램 pigpio-emu

`pigpio-emu [포트] [바인드주소]`. `PigpioEmulatorServer` 를 띄우고 받은 명령을 한 줄씩 찍으며, 100 ms 마다 흉내 낸 서보 위치를 막대(`[────┼─█──] 현재 +15.4° → 목표 +20.0°`)로 그립니다(펄스나 각도가 바뀔 때만). `0.0.0.0` 으로 띄우면 WSL 안의 Qt 판이 Windows 호스트 IP 로 접속할 수 있습니다.

tools/fake_pigpiod.py

파이썬, 표준 라이브러리만

`PigpioEmulatorServer` 를 94줄 파이썬으로 옮긴 것. WSL 을 "가짜 Pi" 로 만들 때 씁니다(`./run-from-wsl.sh fakepi`). `struct.unpack("<IIII")` 로 16바이트를 풀고 `struct.pack("<IIIi")` 로 응답합니다. 접속마다 스레드 하나. WSL2 localhost 포워딩 덕분에 Windows 의 WPF 가 `127.0.0.1:8888` 로 접속하면 이 리눅스 프로세스에 닿습니다.

run-from-wsl.sh · Directory.Build.targets

- **run-from-wsl.sh** — WSL 셸에서 Windows 의 WPF 를 띄우는 실행기. 인수 없음: Release(없으면 Debug) exe 를 `setsid nohup` 으로 띄움(실행 비트 자동 부여). `build`: `dotnet.exe build "$(wslpath -w …sln)"` 후 실행. `emu`: Windows 쪽 pigpio-emu. `fakepi`: 리눅스 쪽 `fake_pigpiod.py`. 원리는 WSL 의 interop(리눅스 셸에서 .exe 를 실행하면 Windows 가 띄움).
- **Directory.Build.targets** — 솔루션 폴더의 모든 프로젝트에 자동 적용되는 MSBuild 조각. `UseWPF=true` 인 프로젝트를 Windows 가 아닌 OS 에서 빌드하면 알기 어려운 `NETSDK1100` 대신 "WPF 는 Windows 에서만 … ./run-from-wsl.sh build 를 쓰세요" 라는 한글 안내로 멈춥니다. 에뮬레이터 프로젝트는 영향 없음.

8. 테스트 프로젝트 — 무엇을 지키는가

`ServoControlWpf.Tests` (xUnit, 53개). 테스트는 "이 규칙이 깨지면 알려 달라" 는 약속입니다. 각 파일이 지키는 규칙을 적었습니다.

파일	수	지키는 규칙	어떻게
`ServoSpecTests`	17	+20° → 1700 µs, −40° → 1100, 범위 밖은 잘림(표 데이터 여러 건), 비대칭 보정, `PulseToAngle` 역변환, 기본값이 공식 HS-311, `IsValid` 가 잘못된 값을 거부(경계값 여러 건)	순수 계산
`PigpioClientTests`	6	요청 바이트가 정확히 `08 00 00 00 12 00 00 00 dc 05 00 00 00 00 00 00`, 음수 res → 한글 메시지(연결 유지), 응답 없음 → 시간 초과 `LinkLost`, 접속 거부 → `LinkLost` + 안내문, `RemotePigpioDriver.Open` 이 HWVER·PIGPV 를 읽고 `Release` 가 펄스 0 을 보냄, 오류 코드 → 문구	인프로세스 가짜 pigpiod(`TcpListener`)
`ServoModelTests`	8	`SetAngle` 성공 시 펄스 출력 + `Changed`, 실패 시 상태 불변, 잘못된 `SetSpec` 거부, 출력 실패 시 이전 사양 유지, 성공 시 각도 클램프·재출력, GPIO 변경 시 이전 핀 Release 후 새 핀 Open, `SetDriver` 가 이전 드라이버를 Release·Dispose, `Release` 가 출력 꺼짐 표시	호출을 기록하는 가짜 드라이버 (`RecordingDriver`)
`EmulatorTests`	3	`ServoModel → RemotePigpioDriver → 에뮬레이터` 전체 경로(Info "Pi 리비전 c03111 · pigpio v79", 펄스 저장), 잘못된 펄스는 실제 pigpio 처럼 거부, 흉내 낸 각도가 시간에 따라 목표로 이동	실제 TCP(127.0.0.1 임의 포트)
`ServoDialTests`	9	마우스 방향 → 값(위 = 0°, ±135° = 끝, 45° → 13°, 빈 구간은 가까운 끝 — 표 데이터 7건), 범위 변경 반영, 1° 단위 맞춤·클램프	STA 스레드에서 컨트롤 생성
`ServoCameraViewTests`	4	연결 해제면 Angle 을 바꿔도 안 움직임 / 연결+출력 켜짐이면 회전 시작 / 출력 꺼짐이면 정지, 켜지는 순간 출발 / 회전 중 해제하면 제자리 정지	STA 스레드
`MainViewModelTests`	5	각도 → 펄스 계산·출력, 펄스 1650 → 각도 15.0° 출력(양방향), 범위 밖 펄스는 min/max 로 잘림, 비대칭 보정에서 왕복, 드라이버 실패 시 이전 값으로 되돌리고 오류 창	가짜 대화상자 (`FakeDialogs`) + 가짜 드라이버
`WslSshTunnelTests`	1	없는 ssh 별명으로 터널을 열면 멈추지 않고 1초 안에 `ServoException` (ssh 오류 문장 포함)	실제 `wsl.exe -e ssh` (WSL 있는 PC)

테스트가 가능한 이유 = 층을 나눈 이유

가짜 드라이버를 꽂을 수 있는 건 `IServoDriver` 인터페이스 덕분이고, 창 없이 ViewModel 을 돌릴 수 있는 건 `IDialogService` 덕분이며, 바이트 열을 검사할 수 있는 건 `PigpioClient` 를 따로 뗐기 때문입니다.

9. 시나리오로 따라가기 — 버튼 하나가 지나가는 길

9-1. [연결] 을 눌렀을 때 (에뮬레이터 모드)

1. **MainWindow**: 버튼의 `Command` → `MainViewModel.ConnectToggleCommand`
2. **MainViewModel.OnConnectToggleAsync**: `IsBusy=true`(버튼 "연결 중…", 설정칸 잠김) → 모드가 Emulator 이므로 `StartEmulatorDriver()`
3. **PigpioEmulatorServer**: `127.0.0.1:8888` 에서 듣기 시작 → **RemotePigpioDriver**("127.0.0.1", 8888, server) 생성
4. `Task.Run(() => driver.Open(18))`: **PigpioClient.Connect** → **Command(HWVER)** → 서버가 0xc03111 응답 → **Command(PIGPV)** → 79 → `Info = "Pi 리비전 c03111 · pigpio v79"`
5. **ServoModel.SetDriver(driver)**: 이전 `SimulatedDriver` Release·Dispose → 교체 → `Changed` 이벤트
6. **MainViewModel.SyncFromModel**: `IsConnected=true` 등 알림 → 버튼 "연결 해제"(초록), 상단에 장치 정보, 왼쪽 카메라 뷰의 `IsConnected` 가 true 로(막이 걷힘)

9-2. 다이얼을 +20° 로 돌렸을 때

1. **ServoDial.OnMouseMove**: 마우스 방향 → `ValueFromPoint` → `Value=20` → 양방향 바인딩이 **MainViewModel.Angle** 에 20 을 set
2. **Angle setter**: `_syncing` 이 false 이므로 `RequestAngle(20)` → 스윕 정지 → **ServoModel.SetAngle(20)**
3. **ServoModel**: `Spec.ClampAngle(20)=20` → `Spec.AngleToPulse(20)=1700` → **RemotePigpioDriver.WritePulse(1700)** → **PigpioClient.Command(8, 18, 1700)** → 16바이트 전송 → res 0
4. 성공했으므로 `Angle=20, IsOutputOn=true` → `Changed`
5. **SyncFromModel**(`_syncing=true`): `Angle`, `Pulse=1700`, `PulseInput`, `IsOutputOn` 등 알림 → 각도 줄 20.0, 펄스 줄 1700, 다이얼 힌트 "1700 µs · 출력 켜짐", **ServoCameraView** 는 목표 20° 를 향해 316 °/s 로 회전 시작, 상단 "가짜 Pi 수신 #3: SERVO gpio=18 1700 µs"

펄스 입력칸에 1650 을 넣으면 2번이 `PulseInput` setter 로 바뀔 뿐입니다: 범위로 자름 → `Spec.PulseToAngle(1650)=15` → `RequestAngle(15)` → 이후 동일.

9-3. 통신이 끊겼을 때

1. **PigpioClient.Command**: 1.5초 안에 응답이 없거나 상대가 끊음 → `Abort()` → `ServoException(linkLost: true)`
2. 예외는 **ServoModel.SetAngle** 을 그대로 통과(상태 불변) → **MainViewModel.RequestAngle** 의 catch → `HandleFailure`
3. `LinkLost` 이므로 `OnDisconnect()`: **ServoModel.SetDriver(new SimulatedDriver())** → 이전 드라이버 `Release`(0.5초 재접속해 펄스 0 시도)·`Dispose`(터널·에뮬레이터 종료)
4. 오류 창: "Pi 응답 없음 (시간 초과) … 연결을 해제했습니다. 다시 [연결]을 눌러 주세요." 화면은 "연결 안 됨" 으로, 카메라 뷰는 그 자리에서 멈추고 막이 덮임

pigpio 가 명령만 거부한 경우(예 −7 PI_BAD_PULSEWIDTH)는 `LinkLost=false` 라 연결을 유지하고 메시지만 보여 줍니다.

9-4. 창을 닫았을 때

1. **App**: `window.Closed` → `ShutdownOnce()` → **MainViewModel.Shutdown()**
2. 스윕 정지 → **AppSettings.Save()**(주소·모드·스크립트 폴더 등) → **WslViewModel.Cancel()**(실행 중인 wsl.exe 종료)
3. **ServoModel.Dispose()** → **RemotePigpioDriver.Dispose()**: `Release()`(SERVO gpio 0 = 힘 빼기) → 소켓 닫기 → 터널/에뮬레이터 Dispose

pigpiod 는 접속이 끊겨도 마지막 펄스를 계속 낸다

그래서 "끊기" 만 하면 서보는 계속 힘을 줍니다. 이 프로그램이 종료·해제·오류 때마다 굳이 펄스 0 을 보내려고 애쓰는(심지어 0.5초 재접속까지 하는) 이유입니다.

10. 어디를 고치면 되나

하고 싶은 일	고칠 곳
다른 서보(예 MG996R)를 쓰고 싶다	화면의 보정값만 바꾸면 됨. 기본값을 바꾸려면 `ServoSpec` 생성자와 `Hs311Official()`
pigpiod 의 다른 명령(예 GPIO 읽기)을 추가	`PigpioClient` 에 상수 추가(명령 번호는 pigpio 문서의 cmd 표) → `RemotePigpioDriver` 에 메서드 → 필요하면 `IServoDriver` 규격 확장 → 에뮬레이터 `Handle()` 에도 흉내 추가
Pi 주소·포트 기본값 변경	`AppSettings` 의 초기값(`Host = "192.168.10.2"` 등). 이미 저장된 JSON 이 우선함
새 버튼(예 "−45°로")	`MainViewModel` 에 `RelayCommand` 하나 + `MainWindow.xaml` 에 버튼 하나. 서보 명령은 `RequestAngle` 을 재사용
스윕 주기·속도 범위	`MainViewModel.SweepIntervalMs` (20 ms) · 생성자의 `Math.Clamp(…, 5, 200)` · XAML 슬라이더 범위
통신 타임아웃	`PigpioClient.DefaultTimeoutMs` (1500) · 터널 대기는 `WslSshTunnel.OpenAsync` 의 `timeoutMs` (10000)
카메라 뷰의 속도·시야각	XAML 에서 `SpeedDegPerSec`, `FieldOfView` 속성을 주거나 `ServoCameraView` 의 기본값
카메라 뷰를 연결 안 돼도 움직이게(원래 동작으로)	`ServoCameraView.IsDriven` 에서 `IsConnected &&` 를 빼고, `OnRender` 의 "연결 안 됨" 블록 제거
WSL 탭에 새 스크립트 버튼	`WslViewModel` 에 `RelayCommand(async () => await RunScriptAsync("x.sh"))` + XAML 버튼
오류 메시지 문구	통신: `PigpioClient.ErrorText` 와 각 `throw new ServoException(…)` · 연결: `MainViewModel.OnConnectToggleAsync` · 터널: `WslSshTunnel`
설정 파일 위치	`AppSettings.FilePath`
색·글꼴	`App.xaml` 의 스타일, `ServoDial` / `ServoCameraView` 맨 위의 `Brush` 상수
리눅스 가짜 Pi 의 동작	`tools/fake_pigpiod.py` 의 `handle()`; .NET 판은 `PigpioEmulatorServer.Handle()` — 둘을 같이 고쳐야 함

고친 뒤 확인하는 순서

`dotnet build ServoControlWpf.sln -c Release` (경고 0 유지) → `dotnet test` (53개) → 에뮬레이터 모드로 [연결] → 다이얼·펄스 입력 → [출력 끄기] → [연결 해제] → 창 닫기. 상단의 "가짜 Pi 수신" 글자가 바뀌면 TCP 로 실제 명령이 나간 것입니다.

11. 용어 사전

용어	뜻 (이 프로젝트에서)
MVVM	Model(데이터·규칙) · View(화면) · ViewModel(둘 사이의 통역). View 는 ViewModel 의 속성에 **바인딩**만 하고 코드를 거의 갖지 않는다
바인딩(Binding)	XAML 의 `{Binding Angle}`. ViewModel 의 `Angle` 이 바뀌면 화면이 저절로 바뀌고 (OneWay), 화면에서 바꾸면 ViewModel 도 바뀐다(TwoWay)
INotifyPropertyChanged	"내 속성이 바뀌었어" 라고 알리는 규격. `ObservableObject` 가 구현
ICommand / RelayCommand	버튼이 누를 수 있는 "동작" 객체. 실행 가능 여부(`CanExecute`)도 가짐
DependencyProperty(의존 속성)	WPF 컨트롤의 바인딩·애니메이션이 되는 특별한 속성. `ServoDial.Value`, `ServoCameraView.Angle` 등
인터페이스(interface)	"이 메서드들을 갖춰라" 는 약속. `IServoDriver`, `IDialogService`. 덕분에 진짜와 가짜를 바꿔 끼울 수 있다
Strategy / Null Object 패턴	같은 규격의 부품을 바꿔 끼우는 것 / "없음" 을 나타내는 빈 부품(`SimulatedDriver`)
예외(Exception)	실패를 호출자에게 던지는 방법. `ServoException.LinkLost` 로 끊김 여부를 함께 전달
DispatcherTimer	UI 스레드에서 주기적으로 코드를 실행하는 타이머(스윕 20 ms, 카메라 16 ms, 상태 줄 4 s)
async / await · Task.Run	오래 걸리는 일(접속, 터널 대기, wsl.exe 실행)을 기다리는 동안 화면이 멈추지 않게 하는 문법
SynchronizationContext.Post	다른 스레드에서 생긴 결과(로그 줄, 에뮬레이터 수신)를 UI 스레드로 넘겨 화면을 안전하게 갱신
TCP / TcpClient / TcpListener	네트워크로 바이트를 주고받는 통로 / 접속하는 쪽 / 기다리는 쪽(가짜 pigpiod)
pigpiod	Pi 에서 GPIO 를 대신 다루는 데몬. 포트 8888 로 16바이트 명령을 받는다
little-endian	숫자를 낮은 바이트부터 쓰는 순서. 1700 = `a4 06 00 00`
WSL2 localhost 포워딩	WSL 안에서 127.0.0.1 에 연 포트를 Windows 의 127.0.0.1 로도 접속할 수 있게 하는 기능. 터널·가짜 Pi 모두 이것에 의존
interop	WSL 셸에서 `.exe` 를 실행하면 Windows 가 그 프로그램을 띄우는 기능. `run-from-wsl.sh` 의 원리
AutomationPeer / UI 자동화	화면 읽기 프로그램·자동 테스트가 컨트롤을 다룰 수 있게 하는 접근성 창구. `ServoDial.DialPeer`
STA 스레드	WPF 요소를 만들 수 있는 특별한 스레드. 컨트롤 테스트가 이것을 따로 만든다

관련 문서: *ServoControlWpf_설계과정.pdf*(왜 이렇게 만들었나, 운용 원리, 검증 결과) · *README.md*(빌드·실행 명령) · 소스 각 파일 맨 위의 주석(역할 요약).